

IT Development Portfolio

Your Showcase Application++

Last Updated: 29 May 2026 (see changelog on Learn)

1 Introduction

At the start of the year, you built a showcase website using static HTML and CSS (and maybe some JavaScript). In the previous block you converted it into an interactive web application. Now is the time to really make your showcase shine! By introducing new elements and updating existing ones, you can transform your website into a complete personal showcase application you can use to promote yourself.

2 Assignment

Given a defined set of themes, you must add to and/or modify a *single web application* in such a way to showcase the implementation of requirements. Your portfolio must adhere to the software development lifecycle. For each of the features implemented that you must have a user story, appropriate designs (both technical and functional), and a subsequent implementation.

Examples of functional design techniques are wireframes with wireflows and flowcharts. For technical design you may make use of class diagrams, and entity relationship diagrams.

For some aspects, you are free to add whatever you want to your website, as long as you can prove that the aspect has been designed and implemented.

2.1 GitHub

Your single portfolio must be managed and committed to the provided GitHub repository. All your design choices must be documented as well. You may make use of the repository's wiki to store your documentation. You will then present your website where you will prove that the conditions have been met.

2.2 Submission & Presentation

You must finalise your entire portfolio by **8 June 2026**. You will then present your portfolio to an assessor in the week of **15 – 19 June 2026**. You must book a session with the assessor. If you do not present your portfolio, you will not be able to earn any points.

2.3 Grading

Each of the requirements/aspects are worth a certain number of points. The total number of points you can achieve is 300. The passmark for the portfolio is set at 201. You can find the grading breakdown in Section 4.2.

3 Assignment Requirements

You can refer to the original PCO Assignment Specification to find out exactly what was required of you in your showcase website.

3.1 Functional & Non-functional Requirements

For this assignment, the following functional and non-functional requirements are required.

3.1.1 Functional Requirements

- As the owner, I want a maintainable blog where I can manage a complete publishing workflow so that my posts remain relevant and consistent.
- As the owner, I want a dashboard where I can track my study programme progress.
- As the owner, I want to manage the professional biographical information so that visitors can easily find more information about me.
- As a visitor, I want to see the progress as well as professional biographical information about the owner so that I can learn about the owner's expertise and progress.

3.1.2 Non-functional Requirements

- The web application must be built with PHP with persistent data.
 - Dependency management and namespaces must conform to PSR-4 by using composer.
 - PHP code style must be consistent with PSR-12.
 - Static code analysis must be performed at PHPStan level 8.
 - Automated testing must be done through a standardised testing framework (see Section 3.4)
 - Docker must be provided for consistent local development and testing (see Section 3.5.1), while deployment must be possible on a standard production environment (see Section 3.5.2)
- The web application must be designed and implemented following established software engineering principles, including clean architecture, appropriate design patterns, and proper database design.
- Usability: The web application must be consistent and easy to use, adhering to established usability heuristics (see Section 3.2)
- Security: The web application must enforce strong authentication and proper authorisation for protected features and be resilient against common web vulnerabilities (see Section 3.3)

3.2 Usability

Usability is the degree to which a software system or interface allows users to achieve their goals effectively, efficiently, and with satisfaction, meaning it is easy to learn, quick to use, minimises errors, supports recovery when mistakes occur, and provides a smooth, intuitive experience through well-designed UI and UX.

Assignment

Create a complete functional design for your application by means of a wireflow. Your functional design must make use of three guidelines of the Nielsen's Usability Heuristics with substantiation:

1. Heuristic 1: Visibility of System Status
2. Heuristic 5: Error Prevention
3. Heuristic 9: Help Users Recognize, Diagnose, and Recover from Errors

Your project must include the complete wireflow (with correct wireframes), including the motivation for each of your chosen usability guidelines. The subsequent implementation must match the submitted wireflow. The fidelity of the wireframes must be high enough to justify and substantiate Nielsen's Heuristics (for example, the wireframe should have colour so that you can discuss errors).

3.3 Security

Security is how well a software system protects its data, resources, and functionality from unauthorised access, misuse, or attacks, ensuring confidentiality, integrity, and availability while allowing legitimate users to safely and reliably perform their intended actions.

Assignment

Design and implement authentication, authorisation, as well as OWASP Top Ten mitigation.

- Allow the user to register and login. Use good password handling practices.
- Restrict certain parts of the application to only certain roles.
- Your application must have clear mitigation of at least three OWASP Top Ten Web

Application Security Risks:

- A01:2025 Broken Access Control
- A05:2025 Injection
- A07:2025 Authentication Failures

Your project must include a complete design, including motivation, of the security practices that you subsequently implemented.

3.4 Testing

Testing is the process of verifying individual units (such as functions or classes) and features (such as user-facing functionality) to ensure they work correctly in isolation and as intended, helping detect defects early and confirm that specific behaviours meet requirements.

Assignment

- Design a test plan for your user stories.
- Create automated tests according to this plan.
 - Make use of integration as well as unit tests within a standardised testing framework for your application.
- The automated tests should have at least 45% code coverage of **app-level** code (as reported by PHPUnit).

3.5 DevOps

Not so long ago, life as a software engineer was relatively simple: developers built software and handed it over to the Operations department, who were responsible for running it, often leading to blame when things went wrong. Over time, it became clear that this approach was ineffective, and modern development now follows the principle of *"You build it, you run it,"* where teams take responsibility for both development and operation. This approach, known as DevOps (a combination of Development and Operations), emphasises shared ownership and reliability, and highlights the importance of creating consistent development environments to avoid issues when deploying applications or collaborating across different systems.

3.5.1 Containers: Local Environment

In this assignment, we use Docker to containerise our application. The assignment is to create a Docker environment that contains a web server with PHP, dependency management tooling, as well as volumes to fully host your application. Your project must provide instructions on the use of the Docker environment in the README.MD file in your Git repository for each of the steps below.

1. The first step is to create a single container that can be started and stopped using manual Docker commands like `docker build` and `docker run` and hosts both the webserver and SQLite database. The application data is stored using Docker volumes. The initial database migration should be included as a separate command.
2. The second step is to add Docker compose. Using Docker compose your container from step 1 should be able to start using a single command.
3. The third step is to separate the database into a separate container that hosts a relational database server like MySQL or PostgreSQL. The webserver and database server containers communicate with each other over a Docker network. Note, to achieve this, you must utilise the repository pattern in the framework to also work with the database server of your choosing.

3.5.2 Deployment (a.k.a. Production)

There is no use in developing an application if you can't deploy it live for everyone to use. For this assignment, you take three steps. Each step is a prerequisite before you can go to the next step. In other words, you can't set up the CI-check before enhancing the readme file.

1. Deploy your application remotely to be fully functional and accessible from anywhere on the internet using a domain name over HTTPS. You may make use of shared hosting platforms, VPS hosting or PaaS hosting.
2. Your project readme must include full instructions on how to set up and release new versions of your application on your selected production environment.
3. To ensure quality before each deployment your application includes several continuous integration (CI) checks that run on GitHub after each push to the main branch:
 - a. PHPStan should report no issues on level 8
 - b. PHPCS should report no issues against the PSR-12 standard
 - c. Deptrac should report no dependency issues

If any of these checks fail the CI step fails and the developer is informed.

3.6 Client-Side APIs

Client-side APIs enable front-end applications to communicate with external services and back-end systems, allowing dynamic data exchange and interaction beyond the browser itself. Using `fetch()`, you can send requests, receive structured data like JSON, and update the user interface in real time.

Assignment

- Create a simple RESTful API of your choice, including at least two GET endpoints with clear request/response formats in JSON.
- Build the API into the backend of your application.
- Develop a front-end page that uses JavaScript/TypeScript **`fetch()`** to communicate with your API, displaying retrieved data dynamically.

3.7 Innovation

Design and implement something unique that has not yet been covered in any course. Have fun with this! Really think outside of the box. Outside of the solar system. Outside of the galaxy.

For your innovative solution, you must provide a full user story, designs, and implementation.

4 Grading Criteria

4.1 Assessability Criteria

The following requirements must be met for your assignment to be graded:

1. The whole assignment is completely committed to the provided GitHub repository.
2. All functional user stories (Section 3.1) are implemented.
3. All documentation is stored on the provided GitHub repository wiki.

4.2 Points Breakdown

1. Functional User Stories	30
a. Functional Designs	
b. Technical Designs	
c. Code Quality & Standards	
2. Usability	50
a. Wireflows	
b. Nielsen Heuristics	
3. Security	50
a. Authentication	
b. Authorisation	
c. OWASP Protection	
4. Testing	50
a. Unit Tests	
b. Integration Tests	
5. DevOps	50
a. Containers: Local Environment	
b. Deployment	
6. Client-Side Scripting	50
7. Innovation	20
Total Points:	300

Pass Mark: 201